Chapter 12 | Exercise solutions with Python

Efficient Execution

This notebook solves Exercises 5–9. It counts a rectangular grid of score tiles, checks outputs across tile sizes, and traces the online softmax state through a masked tile and large scores. It also tests how absolute query positions change a causal mask. Exercises 1–4 and 10–11 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch12_efficient" / "model.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Check an uneven grid

There are five query rows and eleven key columns. Each cell represents one query–key score. A tile contains up to two rows and four columns, so a tile that reaches the bottom or right edge may be smaller. We use zero-based row and column indices.

Query rows	Keys 0–3	Keys 4–7	Keys 8–10
0–1	A · 2 × 4	B · 2 × 4	C · 2 × 3 (partial)
2–3	D · 2 × 4	E · 2 × 4	F · 2 × 3 (partial)
4	G · 1 × 4 (partial)	H · 1 × 4 (partial)	I · 1 × 3 (partial)

Each letter labels one tile. Tile C is partial because the final column group contains only three keys. Tile G is partial because the final row group contains just one query. Tile I has both short edges. The sketch expands the table into individual cells.

```
OUTPUT
                    Key position
Query position   0 1 2 3 | 4 5 6 7 | 8 9 10
       0         A A A A | B B B B | C C C
       1         A A A A | B B B B | C C C
                 --------+---------+-------
       2         D D D D | E E E E | F F F
       3         D D D D | E E E E | F F F
                 --------+---------+-------
       4         G G G G | H H H H | I I I
```

Counting the whole grid gives $5 \times 11 = 55$ scores. Counting by tile also gives $4(8) + 2(6) + 2(4) + 3 = 55$, since every score belongs to exactly one tile. The largest tile holds $2 \times 4 = 8$ scores. Pass the lengths and then the two tile dimensions to `score_workspace` to check these two counts.

```
from examples.ch12_efficient.model import score_workspace

full_count, largest_tile_count = score_workspace(5, 11, 2, 4)
tile_counts = np.outer([2, 2, 1], [4, 4, 3])

np.testing.assert_array_equal(tile_counts.sum(), full_count)
np.testing.assert_array_equal(tile_counts.max(), largest_tile_count)

print("Number of scores in each tile")
print(tile_counts)
print(f"Full grid: {full_count} scores")
print(f"Largest tile: {largest_tile_count} scores")
```

```
OUTPUT
Number of scores in each tile
[[8 8 6]
 [8 8 6]
 [4 4 3]]
Full grid: 55 scores
Largest tile: 8 scores
```

The sum counts every score location in the unmasked grid, even if the algorithm computes those scores at different times. It is also how many elements a single materialized score grid would store. The maximum counts the score elements in one tile at its largest. When tiles are processed sequentially, the algorithm can carry the running softmax state forward and release each tile's score array before moving on. It still visits all 55 scores in this example. Both counts are in elements. Multiplying by the score dtype's bytes per element gives the bytes for the corresponding score array alone. Inputs, outputs, masks, running accumulators, and any separate weight arrays also require space.

6. Change the tile shape

Three queries each have one feature, and the four keys and values each have one feature. Without a mask, every query reads all four keys. Each score is the query–key dot product divided by the square root of the key width. The width here is one, so for a scalar query q_i the scores are $q_i(0, 1, -1, 2)$.

Take the first query, $q_0 = 1$. Its scores are $(0, 1, -1, 2)$. The values at the same positions are $(4, 1, 3, 2)$, so the output is the sum of the four values multiplied by their normalized exponential weights.

$$y_0 = \frac{4 + e + 3e^{-1} + 2e^2}{1 + e + e^{-1} + e^2} \approx 1.9694644227.$$

We will compute a dense reference for all three rows. It subtracts the largest score in each row before exponentiating. This common shift leaves the softmax unchanged, since its factor cancels between the numerator and denominator.

```
from examples.ch12_efficient.model import tiled_attention

tile_queries = np.array([[1.0], [2.0], [3.0]])
tile_keys = np.array([[0.0], [1.0], [-1.0], [2.0]])
tile_values = np.array([[4.0], [1.0], [3.0], [2.0]])

dense_scores = tile_queries @ tile_keys.T
shifted_scores = dense_scores - dense_scores.max(axis=1, keepdims=True)
dense_weights = np.exp(shifted_scores)
dense_weights /= dense_weights.sum(axis=1, keepdims=True)
dense_output = dense_weights @ tile_values

with np.printoptions(precision=10):
    print("Dense output:", dense_output[:, 0])
```

```
OUTPUT
Dense output: [1.9694644227 1.9167694979 1.9575193731]
```

During tiled attention, each query carries a running maximum m, an exponential sum ℓ, and a value-weighted sum u. After a set of scores has been seen, those sums contain e^{s-m} and $e^{s-m}v$ terms respectively. If a new tile raises m, both accumulated sums are rescaled to the new reference before its contributions are added. Every partition processes the same scores and values in this unmasked example. Once all keys have been seen, the ratio u/ℓ therefore gives the same result in exact arithmetic.

Compare each output with the dense reference and with the $(1, 1)$ result. We use float64, absolute tolerance 10^{-12}, and relative tolerance zero, since regrouping a sum can change the last digits in floating-point arithmetic.

```
tile_shapes = [(1, 1), (2, 3), (3, 4)]
tile_outputs = {}
for query_tile, key_tile in tile_shapes:
    shape = (query_tile, key_tile)
    tile_outputs[shape] = tiled_attention(
        tile_queries, tile_keys, tile_values, query_tile, key_tile,
    )

for shape, output in tile_outputs.items():
    np.testing.assert_allclose(
        output, tile_outputs[(1, 1)], rtol=0, atol=1e-12
    )
    np.testing.assert_allclose(output, dense_output, rtol=0, atol=1e-12)
    full, largest = score_workspace(3, 4, *shape)
    error = np.max(np.abs(output - dense_output))
    print(f"Tile {shape} | full grid {full} | largest tile {largest}")
    with np.printoptions(precision=10):
        print(f"  Output {output[:, 0]} | max difference {error:.3g}")
```

```
OUTPUT
Tile (1, 1) | full grid 12 | largest tile 1
  Output [1.9694644227 1.9167694979 1.9575193731] | max difference 2.22e-16
Tile (2, 3) | full grid 12 | largest tile 6
  Output [1.9694644227 1.9167694979 1.9575193731] | max difference 2.22e-16
Tile (3, 4) | full grid 12 | largest tile 12
  Output [1.9694644227 1.9167694979 1.9575193731] | max difference 2.22e-16
```

All three partitions agree within the stated tolerance. They each cover a full grid of twelve scores. With $(1, 1)$, a tile holds just one score. With $(2, 3)$, the two query groups contain two and one rows, and the two key groups contain three and one columns. The four tiles contain $6, 2, 3, 1$ scores respectively, so the largest has six. The $(3, 4)$ tile covers the whole grid and holds all twelve scores at once. These are counts for the score array alone. Tile dimensions can change the size and number of intermediate arrays, so the largest score tile alone cannot tell us the peak memory or which shape runs fastest.

7. Pass through an empty tile

Imagine a query whose first score tile contains two future keys. The dot products in that tile are 4 and 7, and the corresponding values are 8 and 5. The Boolean mask `[[False, False]]` excludes both keys. Neither their scores nor their values should enter the softmax calculation.

For one query, the running state contains a maximum score m, a denominator ℓ, and a weighted numerator u. The denominator adds up the exponential weights of permitted keys. The numerator adds the same weights times the corresponding values. The two sums use scores measured relative to m. The initial state has $m = -\infty$ and $\ell = u = 0$ because it has seen no permitted keys. The state arrays have shapes `[1]`, `[1]`, and `[1, 1]`, respectively. The numerator has an extra axis for its one value feature.

Here the first tile leaves that state unchanged. Internally the helper substitutes $-\infty$ for each masked score. For an empty row it uses zero as the temporary subtraction reference, so exponentiating a masked score gives $\exp(-\infty - 0) = 0$. This avoids subtracting $-\infty$ from itself and still records $-\infty$ as the stored maximum.

```
from examples.ch12_efficient.model import online_update

def empty_online_state():
    return np.array([-np.inf]), np.zeros(1), np.zeros((1, 1))

blocked_state = online_update(
    *empty_online_state(),
    scores=np.array([[4, 7]]),
    values=np.array([[8], [5]]),
    allowed=np.zeros((1, 2), dtype=bool),
)
blocked_m, blocked_ell, blocked_u = blocked_state
np.testing.assert_array_equal(blocked_m, [-np.inf])
np.testing.assert_array_equal(blocked_ell, [0.0])
np.testing.assert_array_equal(blocked_u, [[0.0]])
print(f"Blocked tile  m={blocked_m[0]}, ell={blocked_ell[0]}, "
      f"u={blocked_u[0, 0]}")
print("Output undefined because the denominator is zero.")
```

```
OUTPUT
Blocked tile  m=-inf, ell=0.0, u=0.0
Output undefined because the denominator is zero.
```

The next tile contains one permitted key, with score 2 and value 9. The first finite maximum is $m = 2$. The new key has weight $\exp(2 - 2) = 1$, giving $\ell = 1$ and $u = 9$. The old state contributes zero because it contains no earlier permitted key. The output becomes defined when the first valid tile makes ℓ positive. In a full tiled read, divide only after all tiles have contributed, since later valid keys can change the final ratio.

```
first_key_state = online_update(
    *blocked_state, scores=np.array([[2]]), values=np.array([[9]])
)
first_m, first_ell, first_u = first_key_state
np.testing.assert_array_equal(first_m, [2.0])
np.testing.assert_array_equal(first_ell, [1.0])
np.testing.assert_array_equal(first_u, [[9.0]])
print(f"Valid tile    m={first_m[0]}, ell={first_ell[0]}, "
      f"u={first_u[0, 0]}")
assert np.all(first_ell > 0)
output = first_u / first_ell[:, None]
np.testing.assert_array_equal(output, [[9.0]])
print(f"Output after checking denominator  {output[0, 0]}")
```

```
OUTPUT
Valid tile    m=2.0, ell=1.0, u=9.0
Output after checking denominator  9.0
```

8. Trace a rectangular causal mask

There are two query rows and five key rows. The query row within the current chunk has index $i \in \{0, 1\}$. Its position in the whole sequence is $p = \text{query_start} + i$. The key history starts at zero, so a key with index j is visible exactly when $j \le p$. In particular, the query can read the key at its own position.

All query and key vectors are zero. Every permitted score therefore equals zero, and every permitted key gets the same weight. With r permitted keys, each has softmax weight $\exp(0)/(r\exp(0)) = 1/r$. The output is the mean of the visible values, which makes the effect of the mask easy to check by hand.

Query start	Query row	Absolute position	Readable keys	Output
0	0	0	$[0]$	1
0	1	1	$[0, 1]$	$(1 + 3)/2 = 2$
3	0	3	$[0, 1, 2, 3]$	$(1 + 3 + 7 + 2)/4 = 3.25$
3	1	4	$[0, 1, 2, 3, 4]$	$(1 + 3 + 7 + 2 + 9)/5 = 4.4$

With one query row and two key columns per tile, the key slices are `[0, 1]`, `[2, 3]`, and `[4]`. For absolute position 0, the first tile admits only key 0 and the later tiles admit no keys. For absolute position 3, the first two tiles are fully visible, and the last tile is still in the future. For a chunk starting at position 3, the causal boundary shifts to key columns 3 and 4.

```
from examples.ch12_efficient.model import tiled_attention

offset_q = np.zeros((2, 1))
offset_k = np.zeros((5, 1))
offset_v = np.array([[1], [3], [7], [2], [9]], dtype=float)
key_indices = np.arange(len(offset_k))
expected = {0: [1.0, 2.0], 3: [3.25, 4.4]}

for start, expected_rows in expected.items():
    result = tiled_attention(
        offset_q, offset_k, offset_v, 1, 2,
        causal=True, query_start=start,
    )
    query_positions = start + np.arange(len(offset_q))
    allowed = key_indices[None, :] <= query_positions[:, None]
    means = [offset_v[row, 0].mean() for row in allowed]
    np.testing.assert_allclose(means, expected_rows, atol=1e-12, rtol=0)
    np.testing.assert_allclose(
        result[:, 0], expected_rows, atol=1e-12, rtol=0
    )
    print(f"query_start = {start}")
    for p, row, value in zip(query_positions, allowed, result[:, 0]):
        readable = key_indices[row].tolist()
        print(f"  query {p} reads {readable}, output {value:.2f}")
```

```
OUTPUT
query_start = 0
  query 0 reads [0], output 1.00
  query 1 reads [0, 1], output 2.00
query_start = 3
  query 3 reads [0, 1, 2, 3], output 3.25
  query 4 reads [0, 1, 2, 3, 4], output 4.40
```

The two calls pass the same vectors to the same implementation. Only the absolute position of each query changes. The outputs at start 3 incorporate more of the history because those rows are later in the sequence. If a chunk actually belongs at positions 3 and 4, passing a start of zero wrongly removes three keys from each query. The float64 comparisons above allow an absolute difference of 10^{-12} and use zero relative tolerance.

9. Stress the exponentials (optional)

Give the first score tile $[800, 799]$ and the second tile $[801]$. The corresponding values are $(1, 3)$ and (5). The exponential of a score around 800 exceeds the largest representable float64 number. Online softmax avoids that calculation by subtracting the running maximum before exponentiating.

Write $a = e^{-1}$ and $b = e^{-2}$. The first tile sets $m = 800$. The two shifted scores are 0 and -1, so $\ell = 1 + a$ and $u = 1 + 3a$. When the second tile raises the maximum to 801, multiply both old sums by $\exp(800 - 801) = a$. The new score equals the new maximum and adds weight 1 and weighted value 5. The final state is

$$m = 801, \qquad \ell = a(1 + a) + 1 = 1 + a + b, \qquad u = a(1 + 3a) + 5 = 5 + a + 3b.$$

The direct stable calculation subtracts the final maximum from all three scores at once. It exponentiates the shifted row $(-1, -2, 0)$ and gives the same ratio $(5 + a + 3b)/(1 + a + b)$.

Adding 50 to all the scores also adds 50 to every finite running maximum. The shifted scores within each tile and the difference between the old and new maxima stay the same. Predict that both finite maxima increase by 50 and that both pairs of sums, as well as the final output, agree between the two runs.

```
stress_scores = np.array([[800.0, 799.0, 801.0]])
stress_values = np.array([[1.0], [3.0], [5.0]])

def trace_online(scores):
    state = empty_online_state()
    states = []
    for start, stop in ((0, 2), (2, 3)):
        state = online_update(
            *state, scores[:, start:stop], stress_values[start:stop]
        )
        maximum, denominator, numerator = state
        states.append([maximum[0], denominator[0], numerator[0, 0]])
    return np.array(states)

base_trace = trace_online(stress_scores)
shifted_trace = trace_online(stress_scores + 50)
for shift, trace in ((0, base_trace), (50, shifted_trace)):
    print(f"Score shift +{shift}")
    for tile, (m, ell, u) in enumerate(trace, start=1):
        print(f"  tile {tile}  m={m:.0f}  ell={ell:.9f}  u={u:.9f}")
    assert np.isfinite(trace[-1]).all()
    assert trace[-1, 1] > 0
```

OUTPUT

```
Score shift +0
  tile 1  m=800  ell=1.367879441  u=2.103638324
  tile 2  m=801  ell=1.503214724  u=5.773885291
Score shift +50
  tile 1  m=850  ell=1.367879441  u=2.103638324
  tile 2  m=851  ell=1.503214724  u=5.773885291
```

```
def direct_stable_output(scores):
    weights = np.exp(scores - scores.max(axis=1, keepdims=True))
    return (weights @ stress_values) / weights.sum(axis=1, keepdims=True)

a, b = np.exp([-1.0, -2.0])
hand_sums = [[1 + a, 1 + 3*a], [1 + a + b, 5 + a + 3*b]]
np.testing.assert_array_equal(base_trace[:, 0], [800.0, 801.0])
np.testing.assert_array_equal(shifted_trace[:, 0] - base_trace[:, 0], 50)
np.testing.assert_allclose(
    base_trace[:, 1:], hand_sums, atol=1e-12, rtol=0
)
np.testing.assert_allclose(
    shifted_trace[:, 1:], base_trace[:, 1:], atol=1e-12, rtol=0
)
online_outputs = [trace[-1, 2] / trace[-1, 1]
                  for trace in (base_trace, shifted_trace)]
direct_outputs = [direct_stable_output(stress_scores + shift)[0, 0]
                  for shift in (0, 50)]
np.testing.assert_allclose(
    online_outputs, direct_outputs, atol=1e-12, rtol=0
)
np.testing.assert_allclose(
    online_outputs[0], online_outputs[1], atol=1e-12, rtol=0
)
for shift, online, direct in zip((0, 50), online_outputs, direct_outputs):
    print(f"+{shift:<2}  online {online:.12f}  direct {direct:.12f}")
```

```
OUTPUT
+0   online 3.841024969440  direct 3.841024969440
+50  online 3.841024969440  direct 3.841024969440
```

The stored maxima move from $(800, 801)$ to $(850, 851)$. The denominator and numerator after each tile agree across the two runs, and the final states are finite. Both outputs match the directly stabilized calculation within an absolute tolerance of 10^{-12} with zero relative tolerance in float64. The same comparison works without forming an exponential of any of the original large scores.